

HTML & CSS – Concepts de bases

HTML et CSS sont les deux langages fondamentaux du web. HTML sert à structurer le contenu d'une page (titres, textes, listes, images...) : c'est le « QUOI ». CSS permet d'en contrôler l'apparence (couleurs, tailles, espacements, mise en forme) : c'est le « COMMENT ». Derrière chaque page Web, il y a du HTML et du CSS. La conception de pages web nécessite de la rigueur, de l'organisation, de la créativité et un certain sens de l'esthétique et de la lisibilité.

Comprendre le rôle du HTML et du CSS

HTML : structurer le contenu d'une page

HTML sert à donner du sens au contenu (titre, texte, liste, image...). On repère des balises ouvrantes et des balises fermantes. Ce n'est pas un langage de programmation, c'est un langage de balisage.

```
<h1>Mon titre</h1>
<p>Ceci est un paragraphe.</p>
<ul>
  <li>HTML</li>
  <li>CSS</li>
</ul>
```

Mon titre

Ceci est un paragraphe.

- HTML
- CSS

CSS : mettre en forme le contenu d'une page

CSS modifie l'apparence des éléments HTML. Ce n'est pas un langage de programmation. Appliquons ces styles au HTML ci-dessus :

```
h1 {
  font-size: 16px;
}

p {
  color: blue;
}
```

Mon titre

Ceci est un paragraphe.

- HTML
- CSS

Le squelette minimal d'un fichier HTML

Ce squelette est indispensable pour que la page soit comprise par le navigateur. Attention aux indentations, très utiles pour relire son propre code... Chaque balise a un rôle précis. Elles sont imbriquées les unes dans les autres, mais jamais croisées.

```
<!doctype html>                                <!-- ce document est codé en HTML5 -->
<html lang="fr">                                  <!-- la page ; son contenu est en français -->
  <head>                                           <!-- l'en-tête de la page (paramétrage) -->
    <meta charset="utf-8">                         <!-- données liées aux caractères affichés -->
    <title>Titre de l'onglet</title>              <!-- le titre de l'onglet dans le navigateur -->
  </head>                                          <!-- fermeture de l'en-tête -->

  <body>                                           <!-- le corps de la page (ce qui s'affiche) -->
    <p>Texte visible</p>                          <!-- un paragraphe -->
  </body>                                         <!-- fermeture du corps de la page -->
</html>                                          <!-- fin de la page -->
```

Erreurs fréquentes

- Confondre les rôles du HTML et du CSS.
- Croiser les balises dans le fichier HTML.
- Écrire du contenu dans le `<head>` ou des paramètres dans le `<body>` (souvent, `<title>`).
- Oublier une balise fermante ou le `/` qu'elle contient.
- Mal indenter son code.

Exercices

Lire et comprendre du code : répondre aux questions.

```
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <title>Présentation</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Mon site personnel</h1>
    <p>Bienvenue sur mon site.</p>
    <ul>
      <li>Présentation</li>
      <li>Projets</li>
      <li>Contact</li>
    </ul>
  </body>
</html>
```

```
body {
  font-family: Arial;
}

h1 {
  color: red;
  border-color: red;
  text-align: center;
}

p {
  font-size: 18px;
}

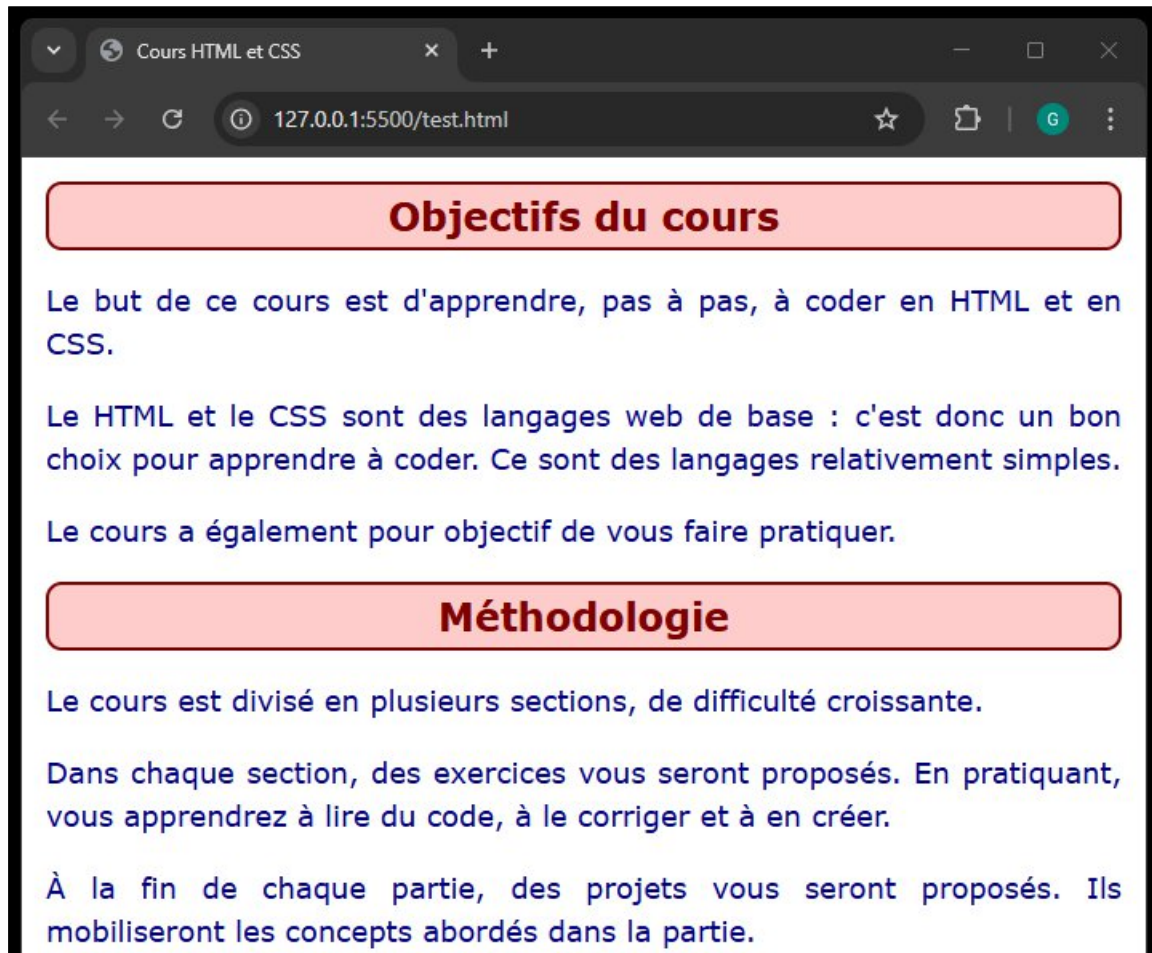
ul {
  background-color: blue;
}
```

- Dans ce projet, où est le fichier :
 - index.html ?
 - style.css ?
- Quel est le titre de la page ?
- Quel est le rôle de la balise `<h1>` ?
- Que permet de faire la règle CSS appliquée à `h1` ?
- À votre avis, quel est l'aspect de l'élément `<ul>` ?

Lire et écrire du code : corriger les erreurs du code HTML suivant.

```
<!doctype html>
<html>
  <head>
  <body>
</head>
  <title>Page</title>
  <h1 style="color: red;">Mon titre
<p>
  <font size="5">Bienvenue sur mon site</font>
<p>
  <p style="margin-left: 20px;">
    Projets
  </p>
<p>Contact</p>
  </body>
</html>
```

Écrire du code : coder le contenu de la page ci-dessous (négliger la mise en forme).



Organiser le contenu avec HTML

Titres et paragraphes

Les titres structurent le texte, les paragraphes contiennent le contenu. L'ordre doit être cohérent.

```
<h1>Titre de niveau hiérarchique 1</h1>  <!-- conseil : un seul par page -->
<h2>Titre de niveau hiérarchique 2</h2>
<h3>Titre de niveau hiérarchique 3</h3>
<h4>Titre de niveau hiérarchique 4</h4>
<h5>Titre de niveau hiérarchique 5</h5>
<h6>Titre de niveau hiérarchique 6</h6>
<p>Texte du paragraphe. Il peut être très long.</p>
```

Listes ordonnées et non ordonnées

Les listes structurent clairement les informations.

```
<ul>                                <!-- liste non ordonnée (liste à puces) -->
  <li>Accueil</li>
  <li>Contact</li>
</ul>

<ol>                                <!-- liste ordonnée (liste numérotée) -->
  <li>Étape 1</li>
  <li>Étape 2</li>
</ol>
```

Liens hypertexte

Les liens permettent de naviguer entre les pages. `target="_blank" rel="noopener noreferrer"` permet d'ouvrir un nouvel onglet.

```
<a href="page2.html">Aller à la page 2 du site</a>
<a href="https://example.com" target="_blank" rel="noopener noreferrer">Autre site</a>
```

Fichiers média

Les médias enrichissent le contenu : images, audio et vidéo peuvent être insérés.

```

<audio controls src="son.mp3"></audio>
<video controls>
  <source src="video.mp4" type="video/mp4"/>
</video>
```

Balises de structure

Elles organisent la page en grandes zones.

```
<header></header> <!-- l'en-tête de la page -->
<main></main> <!-- le corps principal de la page -->
<section></section> <!-- une partie cohérente, avec un titre et du contenu -->
<article></article> <!-- un article au contenu indépendant -->
<nav></nav> <!-- un élément permettant la navigation ; contient des liens -->
<aside></aside> <!-- un élément placé "à côté" de la page : menu, pubs, ... -->
<footer></footer> <!-- le pied de page -->
```

On trouve des `<article>` avec plusieurs `<section>` (page wikipedia) et inversement (site d'info).

Erreurs fréquentes

- Utiliser plusieurs `<h1>` dans la même page.
- Sauter des niveaux de titres.
- Confondre `<head>` et `<header>`.
- Créer un lien sans `href` ou mal nommer le fichier vers lequel pointe le lien.
- Oublier `alt` sur les images.
- Utiliser `<div>` à la place des balises de structure.

Exercices

Lire du code : représenter et légender tous les éléments sous forme de rectangles.

```
<header>
  <h1>Club Nature</h1>
</header>
<main>
  <h2>Activités</h2>
  
  <p>Le club propose des sorties le mercredi.</p>
  <ul>
    <li>Observation</li>
    <li>Jardinage</li>
  </ul>
</main>
<footer>
  <p>Copyright Club Nature</p>
</footer>
```

Lire et écrire du code : corriger les erreurs du bloc de code suivant.

```
<body>
  <head>
    <h2>Mon titre</h2>
    <section>
      <a href="contact.html">Me contacter</a>
      <a href="projets.html">Projets</a>
    </section>
  </head>

  <article>
    <h1>Titre de l'article
    <h2>Sous titre</h2>
    <section>
      <p>...</p>
      <p>...</p>
    </section>
    <section>
      <h2>Sous titre</h2>
      <p>...</p>
      <p>...</p>
    </section>
  </article>
</article>...</article>

  <footer>
    <a href="contact.html">Me contacter</a>
  </footer>
</body>
```

Écrire du code : coder en HTML le contenu demandé.

L'AS du collège veut créer une page qui contient :

- Un en-tête avec le titre « Association Sportive du Collège », et un logo (`logo.png`)
- Une partie principale avec une section sur plusieurs sports : danse, ultimate, échecs.
- Pour chaque sport : le nom, une photo (`le_sport.jpg`), la liste des 3 prochaines dates.
- Un pied de page avec un lien vers l'adresse mail du collège (`mailto:contact@as-college.fr`).

Mettre en forme avec CSS

Appel d'un fichier CSS par un fichier HTML

Ce lien se fait dans `<head>`. Sans ce lien, aucun style n'est appliqué.

```
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

Syntaxe d'une règle CSS

Une règle CSS est composée d'un sélecteur, puis d'un bloc de couples « propriété: valeur ».

```
p {
  color: black;
  font-size: 14px;
}
```

Sélecteurs simples

- Sélecteur de type : on sélectionne tous les éléments de type `<p>` par exemple.
- Sélecteur de classe : on sélectionne tous les éléments de classe `class="info"` par exemple.
- Si une propriété est définie deux fois, la classe est prioritaire sur le type.

```
<p class="info">Du texte</p>
<p class="info">Encore du texte</p>
```

```
p {
    color: black;
}

.info {
    color: green;
}
```

Texte

On peut modifier de nombreux paramètres sur le texte, comme :

```
p {
    font-family: Arial, sans-serif; /* la police d'écriture */
    font-size: 16px;                /* la taille */
    line-height: 1.5;              /* l'interligne */
    text-align: justify;            /* l'alignement */
}
```

Bordures et marges

Les éléments sont des boîtes rectangulaires. Pour chaque boîte, on peut modifier par exemple :

```
section {
    margin: 20px;          /* la marge extérieure */
    padding: 10px;         /* la marge intérieure */
    border: 2px solid black; /* la bordure */
}
```

Arrière-plan

Le fond peut être coloré ou illustré.

```
header {
    background-color: lightgray; /* on met une couleur */
}

section {
    background-image: url("fond.jpg"); /* on met une image (plus difficile) : */
    background-size: cover;            /* cela nécessite des paramètres en plus */
}
```

Erreurs fréquentes

- Oublier de lier le fichier CSS au fichier HTML.
- Oublier le ; à la fin du couple « propriété: valeur ».
- Confondre `margin` (marge extérieure) et `padding` (marge intérieure).
- Utiliser une classe en CSS sans l'avoir utilisée en HTML.
- Chercher à mettre en forme avant d'avoir structuré la page.
- Multiplier les styles différents sans cohérence.

Exercices

Lire du code : indiquer la couleur et la taille de police de chaque élément (par défaut, elle est de 16px).

```
<h2 class="cl_1 cl_2">Titre 1</h2>

<p class="cl_2">Paragraphe 1</p>
<p>Paragraphe 2</p>
<p class="cl_1">Paragraphe 3</p>

<h2 class="cl_1">Titre 2</h2>

<p>Paragraphe 4</p>
<p class="cl_1">Paragraphe 5</p>
<p>Paragraphe 6</p>
```

```
h2 {
  color: red;
  font-size: 24px;
}

p {
  color: blue;
}

.cl_1 {
  color: green;
}

.cl_2 {
  font-size: 18px;
}
```

Lire et écrire du code : corriger les erreurs du code suivant.

```
p {
  color blue
  font-size: 16px
}

.info {
  font-size: 18px;
  color: green
  padding: 10px 5px;
}

titre {
  margin 20 px;
  background color: orange;
}
```

Écrire du code : coder en CSS pour obtenir le résultat demandé.

- une marge extérieure de 20 px pour la section,
- un texte justifié, de taille 20px, dans les paragraphes,
- une marge intérieure de 10 px pour le titre,
- un titre centré, de taille 28px,
- la police "sans-serif" dans toute la section,
- une bordure noire de 2 px pour le deuxième paragraphe seulement.

```
<section>
  <h2>Mon titre</h2>
  <p>
    Mon premier paragraphe très long qui prend plusieurs lignes pour qu'on puisse
    vérifier si la mise en forme est correcte.
  </p>
  <p>Mon second paragraphe plus court, c'est nettement moins pénible.</p>
</section>
```

Énoncé

Vous devez coder une page HTML contenant votre CV, puis la mettre en forme avec une feuille CSS.

- Il y a un en-tête contenant :
 - un titre « Recherche de stage de découverte 3ème »,
 - une photo d'identité,
 - vos noms et prénoms.
- Il y a plusieurs sections avec un titre, et une liste d'éléments :
 - vos coordonnées (commune de résidence, téléphone, mail ; précisez si vous avez le BSR) ; des icônes sont possibles en utilisant une petite image,
 - votre formation (lieu et années),
 - vos projets réalisés (pas seulement en codage),
 - vos compétences (vos points forts : rédaction, relationnel, ...),
 - votre maîtrise des langues (inclure les langages informatiques),
 - vos centres d'intérêt.

Conseils de réalisation

AVANT DE CODER :

- Dessiner le projet sur papier (dessiner et nommer les rectangles) ; vous pouvez utiliser des couleurs pour mieux vous rendre compte du rendu.
- Réfléchir à la structure à donner à votre projet (quelles balises, comment les imbriquer).

ORGANISER LA PAGE EN HTML :

- Placer les balises adaptées.
- Ajouter les éléments de texte.
- Insérer les images nécessaires (photo, icônes).

METTRE EN FORME EN CSS :

- Coder la mise en forme générale avec des règles simples.
 - police d'écriture
 - taille de la police des différents éléments
 - couleur des titres et du texte
- Structurer visuellement les différentes parties en travaillant les boîtes.
 - marges extérieures et intérieures
 - bordures
 - arrière-plans
- Améliorer la lisibilité
 - alignements
 - ajustement des espaces (aérer)
 - choix de couleurs esthétiques (éviter les couleurs « agressives » : rouge, fluo, ...).

HTML & CSS – Concepts intermédiaires

Structurer un projet plus complexe

Lisibilité du code HTML

Il est essentiel que le HTML soit correctement indenté et aéré pour être bien lisible. Des commentaires bien placés permettent de repérer les grandes parties du document ; ils sont bornés par `<!--` et `-->`. Cela permet de gagner en lisibilité, même pour un fichier court. Exemple :

```
<body>

  <!-- En-tête du site -->
  <header>
    <h1>Mon site</h1>
    <nav>
      <a href="index.html">Accueil</a>
      <a href="projets.html">Projets</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <!-- Contenu principal -->
  <main>
    <section>
      <h2>Présentation</h2>
      <p>Bienvenue sur mon site personnel.</p>
    </section>

    <section>
      <h2>Projets</h2>
      <p>Voici quelques projets réalisés.</p>
    </section>
  </main>

  <!-- Pied de page -->
  <footer>
    <p>Contact : exemple@email.fr</p>
  </footer>

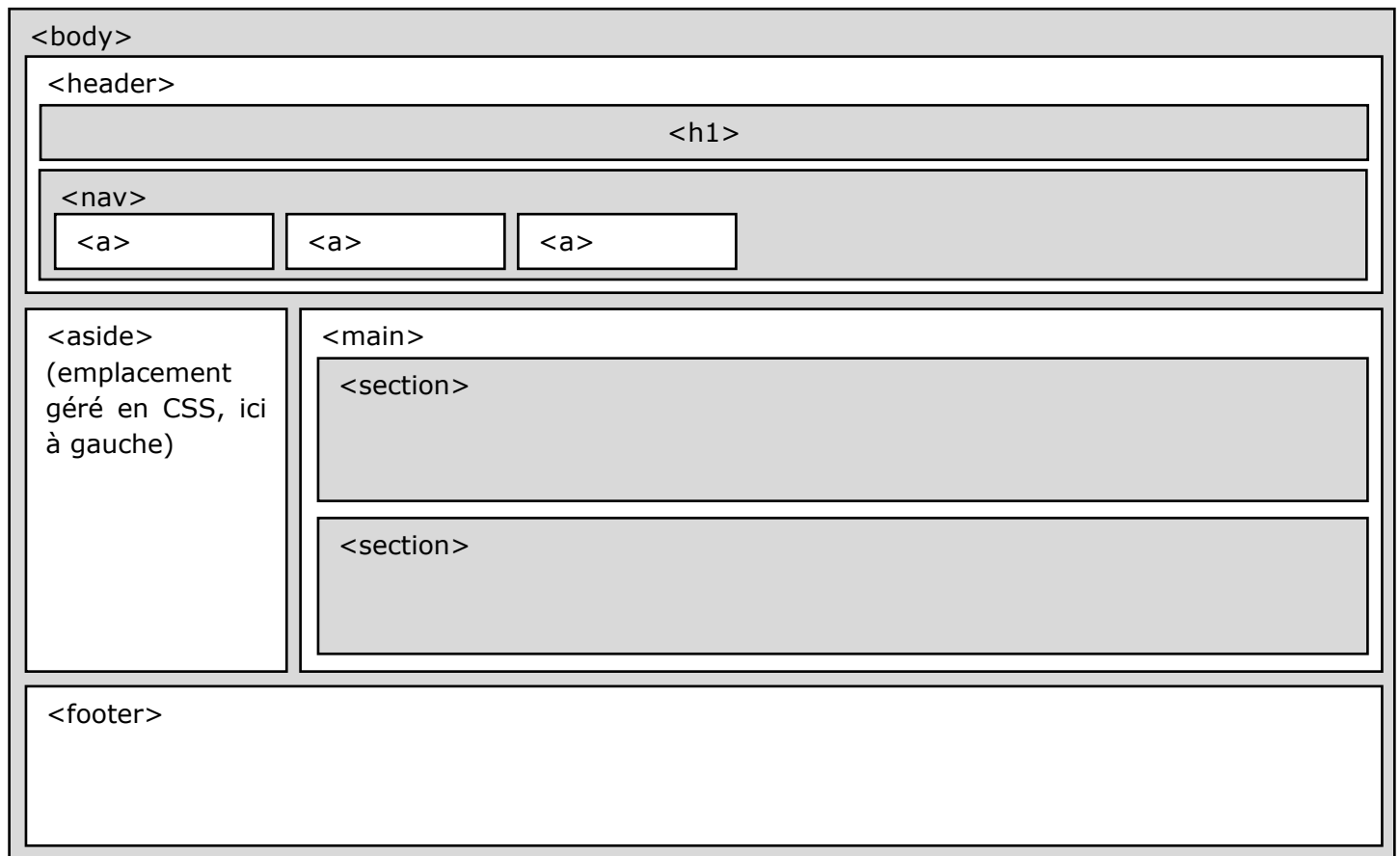
</body>
```

```
<body>
<header><h1>Mon site</h1><nav><a href="index.html">Accueil</a><a
href="projets.html">Projets</a><a href="contact.html">Contact</a></nav></header>
<main>
<section><h2>Présentation</h2><p>Bienvenue sur mon site personnel.</p></section>
<section><h2>Projets</h2><p>Voici quelques projets réalisés.</p></section>
</main>
<footer><p>Contact : exemple@email.fr</p></footer>
</body>
```

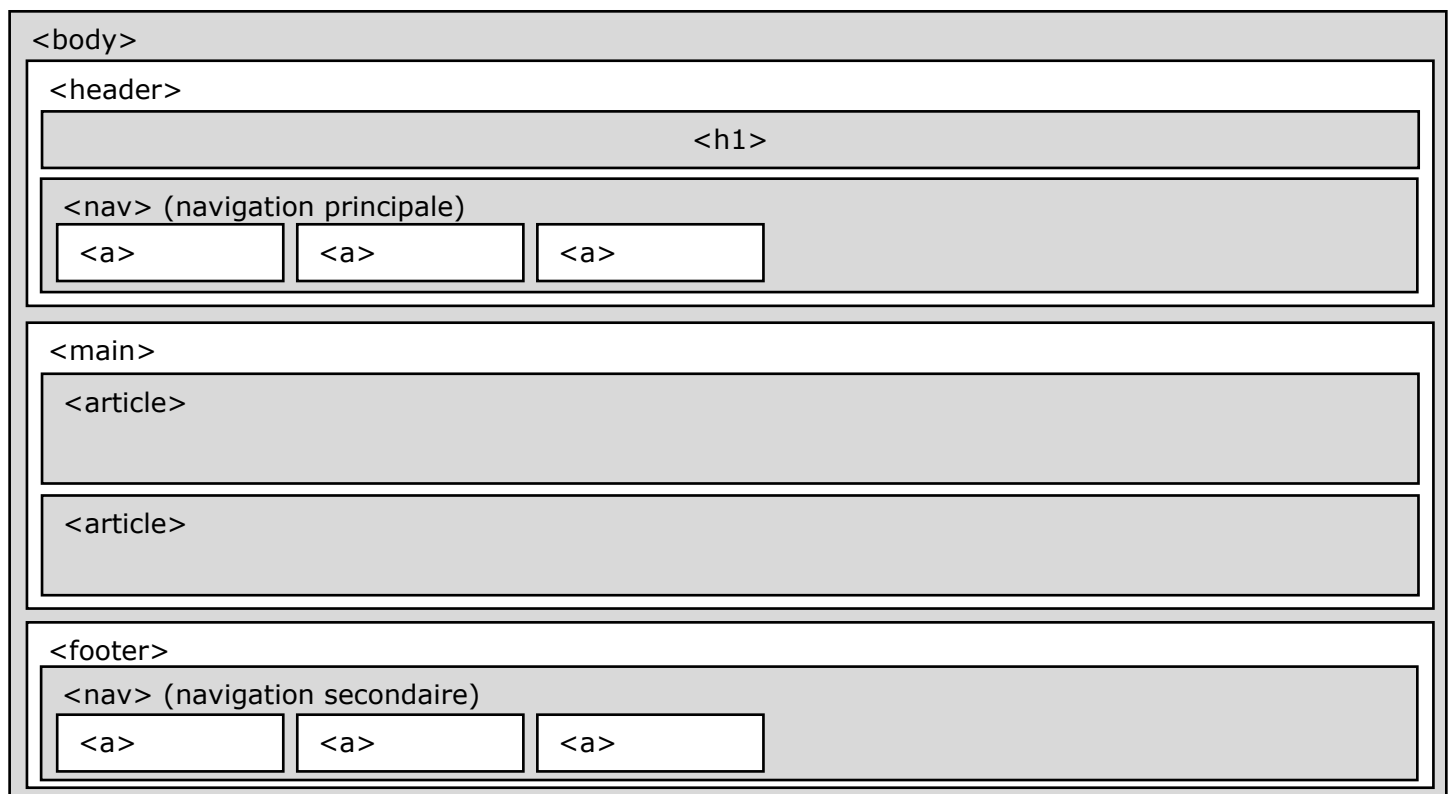
Organisation logique d'une page longue

L'utilisation des balises structurantes est essentielle pour découper le code en parties clairement identifiées et identifiables, avec un rôle précis. La mise en page se fait toujours en CSS. Exemples d'architecture d'une page longue :

- Exemple 1 - avec `<aside>` :



- Exemple 2 - `<nav>` dans le `<footer>` :



La navigation dans un site

Les éléments de navigation sont regroupés dans la balise `<nav>`, sous forme de liens (`<a>`). Pour être bien identifiable par l'utilisateur, la navigation doit être claire et identique sur toutes les pages du site.

- Navigation principale : `<nav>` est placé dans le `<header>` (horizontale) ou dans un `<aside>` (latérale) ; on gère toujours la disposition des éléments via le CSS.
- Navigation vers des liens externes ou navigation utilitaire : la balise `<nav>` est dans le `<footer>`.
- Navigation au sein d'une même page longue : les liens ciblent des `id` (ex : un `id` par `<section>`).
- Navigation hiérarchisée : pour des menus et sous-menus ; il peut y avoir plusieurs `<nav>` imbriqués.

Bon usage de `<div>` et `<span>`

`<div>` et `<span>` sont des balises sans signification (on dit qu'elles sont « neutres »). Ils sont utilisés quand aucune autre balise n'est pertinente ou lors de la mise en forme du site en CSS.

- `<div>` est affiché comme un élément bloc (sous l'élément qui le précède).
- `<span>` est affiché comme un élément inline (sur la même ligne que l'élément qui le précède).

```
<div>
  <div>texte 1</div>
  <div>texte 2</div>
</div>
<div>
  <span>texte 3</span>
  <span>texte 4</span>
</div>
```

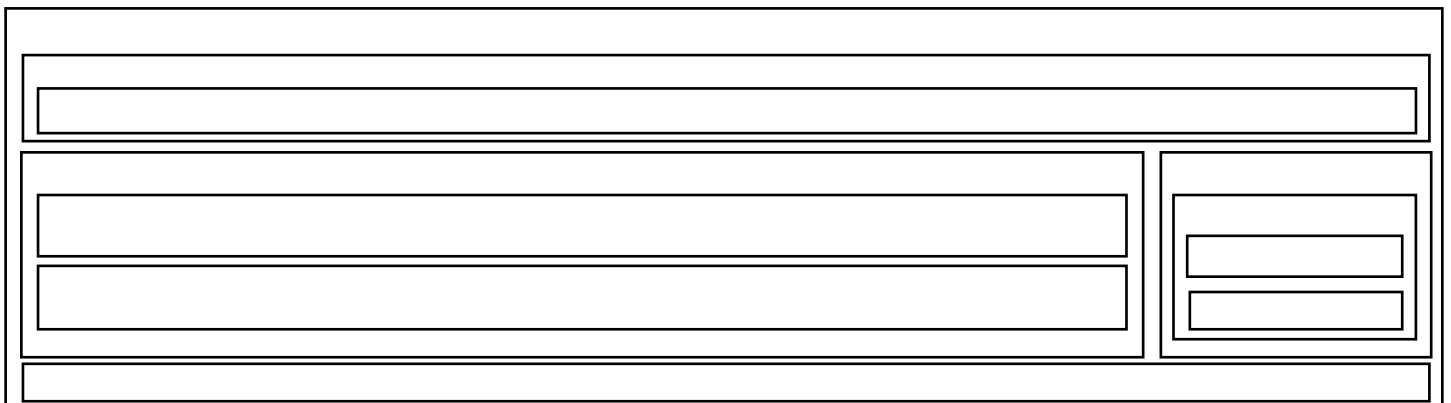
texte 1
texte 2
texte 3 texte 4

Erreurs fréquentes

- Code non indenté ou mal indenté.
- Absence de titres dans une page longue.
- Navigation placée n'importe où dans la page.
- Dupliquer la navigation différemment sur chaque page.
- Utiliser uniquement des `<div>` pour structurer toute la page.
- Commentaires trop nombreux ou inutiles.

Exercices

Lire du code : légender les blocs dans cette page.



Lire et écrire du code : nettoyer et commenter le code suivant pour le rendre lisible.

```
<header><h1>Mon site</h1></header>
<nav><a href="index.html">Accueil</a><a href="contact.html">Contact</a></nav>
<main><section><p>Présentation</p><p>Texte...</p></section></main>
```

Écrire du code : coder pour obtenir le rendu demandé, sans coder de CSS.

Coder lisiblement la navigation d'un mini-site de 4 pages (accueil, activités, historique, contact).

Utiliser les sélecteurs CSS intermédiaires

Lisibilité du code CSS

Un fichier CSS lisible est plus simple à comprendre et à corriger. Il évite aussi les doublons et donc le risque d'erreurs difficiles à corriger. On regroupe les règles par thème et on commente les grandes parties. On utilise `/*` et `*/`. Les deux morceaux de code suivants affichent le même résultat.

```
/* ===== Styles généraux ===== */
body {
  font-family: sans-serif;
  margin: 0;
  padding: 0;
}

/* ===== En-tête et navigation ===== */
header {
  background-color: lightgray;
  padding: 20px;
}

nav a {
  margin-right: 15px;
  text-decoration: none;
  color: black;
}

/* ===== Contenu principal ===== */
section {
  margin: 20px;
  padding: 10px;
  border: 1px solid black;
}

/* ===== Pied de page ===== */
footer {
  text-align: center;
  font-size: 14px;
}
```

```
body{font-family:sans-serif;margin:0;padding:0;}
header{background-color:lightgray;padding:20px;}
nav a{margin-right:15px;text-decoration:none;color:black;}
section{margin:20px;padding:10px;border:1px solid black;}
footer{text-align:center;font-size:14px;}
```

Sélecteurs multiples

Un sélecteur multiple permet d'appliquer les mêmes règles à plusieurs éléments.

```
h1, h2, h3 { /* les éléments de type h1, h2 et h3 sont concernés */
  color: darkred;
}
```

Sélecteurs combinés

Il est possible de combiner les éléments selon leur contexte. Cela évite de créer des classes.

- Sélecteur de descendant :

```
section p { /* les paragraphes inclus dans les sections sont concernés */
  color: blue;
}
```

- Sélecteur de parent direct :

```
section > p { /* les paragraphes "enfant" des sections sont concernés */
  color: green;
}
```

- Combinaison type + classe :

```
p.important { /* les paragraphes de classe .important sont concernés */
  font-weight: bold;
}
```

Spécificité simple et ordre d'application des règles

- La règle la plus spécifique est prioritaire,
- À spécificité égale, la dernière règle écrite est appliquée.

```
p {
  color: black;
}

.section p {
  color: blue;
}
```

Erreurs fréquentes

- Multiplier les sélecteurs trop complexes.
- Oublier que l'ordre des règles a une importance.
- Réécrire plusieurs fois la même règle inutilement.
- Ne pas indenter ou commenter le CSS.

Exercices

Lire du code : indiquer la couleur de chaque paragraphe.

```
<section class="bloc">
  <p>Paragraphe 1</p>
  <p class="important">Paragraphe 2</p>
</section>
```

```
p {
  color: black;
}

.bloc p {
  color: blue;
}

p.important {
  color: red;
}
```

Lire et écrire du code : nettoyer le code suivant pour le rendre lisible.

```
h1,h2{color:blue}
.section p{font-size:16px}
.section>p{color:green}
```

Écrire du code : coder pour obtenir le rendu demandé, sans coder de HTML.

- Tous les titres et des paragraphes de couleur bleue pour sections.
- Les paragraphes encadrés en rouge dans la section `.article`,
- Un CSS lisible et commenté,
- Le titre de la section de classe `.sous` centré.

```
<section class="article">
  <h2>Titre</h2>
  <p>Paragraphe 1</p>
  <p>Paragraphe 2</p>
</section>
<section class="sous">
  <h3>Sous-titre 1</h3>
  <p>Paragraphe 3</p>
  <p>Paragraphe 4</p>
</section>
<section>
  <h3>Sous-titre 2</h3>
  <p>Paragraphe 5</p>
  <p>Paragraphe 6</p>
</section>
```

Maîtriser des cas particuliers de mise en forme

Coins arrondis, ombre, transparence

- La propriété `border-radius` arrondit les coins d'un élément.

```
section {
  border: 2px solid black;
  border-radius: 10px;
}
```

- L'ombre (`box-shadow`) donne du relief à un élément. Syntaxe : décalage horizontal, décalage vertical, flou, couleur. Cela met en valeur une zone (carte, encadré...) avec un effet de volume.

```
section {
  border: 1px solid black;
  box-shadow: 4px 4px 8px gray;
}
```

- On peut rendre une couleur partiellement transparente avec `rgba()`. Les trois premiers nombres = rouge, vert, bleu (0 à 255). Le dernier = transparence (0 à 1) (0 = transparent, 1 = opaque).

```
header {
  background-color: rgba(0, 0, 0, 0.2);
}
```

L'animation des liens avec `:hover`

Il est possible de modifier les liens au survol de la souris avec `:hover`.

```
a {
  background-color: white;
}

a:hover {
  background-color: blue;
}
```

Les tableaux

Un tableau se code en HTML avec plusieurs éléments imbriqués : `<table>` (le tableau), `<tr>` (une ligne), `<th>` (une cellule d'en-tête), `<td>` (une cellule normale). Sa mise en forme se fait en CSS.

```
<table>
  <tr>
    <th>Jour</th>
    <th>Horaire</th>
  </tr>
  <tr>
    <td>Lundi</td>
    <td>17h - 20h</td>
  </tr>
</table>
```

```
table {
  border-collapse: collapse; /* Évite
d'avoir des doubles bordures */
}

th, td {
  border: 1px solid black;
  padding: 8px;
}
```

Erreurs fréquentes

- Mettre des arrondis trop grands ou sur tous les éléments.
- Utiliser des ombres trop fortes (page « chargée » et illisible).
- Confondre `opacity` (qui rend un élément transparent) et `rgba()` (qui rend une couleur transparente).
- Oublier `border-collapse` ou `padding` sur un tableau.

Exercices

Lire du code : dessiner le rendu.

```
<section class="carte">
  <h2>Titre</h2>
  <p>Texte...</p>
</section>
```

```
.carte {
  border: 2px solid black;
  border-radius: 12px;
  box-shadow: 6px 6px 10px gray;
  background-color: rgba(255, 0, 0, 0.2);
}
```

Lire et écrire du code : corriger les erreurs du code suivant.

```
section {
  border radius: 10px
  box-shadow: 4px 4px grey;
  background-color: rgba(0, 0, 0, 2);
}
```

Écrire du code : coder pour obtenir le rendu demandé, sans coder de HTML.

- une bordure noire fine sur le tableau et les cellules,
- des cellules aérées,
- une seule bordure aux cellules,
- une première ligne mis en valeur (fond gris clair + texte en gras),
- des coins légèrement arrondis pour le tableau.

```
<table>
  <tr>
    <th>Sport</th>
    <th>Jour</th>
  </tr>
  <tr>
    <td>Danse</td>
    <td>Lundi</td>
  </tr>
  <tr>
    <td>Ultimate</td>
    <td>Mercredi</td>
  </tr>
</table>
```

Utiliser les Flexbox

Principe du modèle Flexbox

Le modèle `Flexbox` permet d'organiser facilement des éléments en ligne ou en colonne, et de gérer leur alignement et leur espacement. `Flexbox` agit sur un conteneur et sur les éléments qu'il contient. Tous les éléments à l'intérieur du conteneur (ici, les éléments de classe `.item`) deviennent des éléments flexibles.

```
<div class="conteneur">
  <div class="item">...</div>
  <div class="item">...</div>
  <div class="item">...</div>
</div>
```

```
.conteneur {
  display: flex;
}
```

Propriétés du conteneur

Il y a plusieurs propriétés associées au modèle `Flexbox`.

```
.container {
  display: flex;           /* déclaration du Flexbox */
  flex-wrap: wrap;        /* retour à la ligne */
  flex-direction: row;     /* direction de l'axe principal */
  justify-content: center; /* alignement selon l'axe principal */
  align-items: center;     /* alignement selon l'axe secondaire */
  gap: 25px;              /* espace entre les éléments enfants */
}
```

Erreurs fréquentes

- Oublier `display: flex` sur le conteneur.
- Appliquer les propriétés `Flexbox` aux éléments enfants au lieu du conteneur.
- Mélanger `justify-content` et `align-items`.
- Utiliser `Flexbox` pour tout, même quand une mise en page simple suffit.
- Ne pas tester le rendu avec plusieurs éléments.

Exercices

Lire du code : décrire la disposition des éléments codés.

```
<div class="menu">
  <div>Accueil</div>
  <div>Projets</div>
  <div>Contact</div>
</div>
```

```
.menu {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

Lire et écrire du code : corriger le code suivant pour obtenir le rendu demandé.

- Les éléments sont alignés en haut.
- Les éléments sont centrés horizontalement.
- Les éléments sont espacés de 20 pixels.
- Les éléments vont à la ligne s'ils sont trop nombreux.

```
.container {
  justify-content: flex-start;
  align-items: flex-start;
  gap: 20
}
```


Écrire du code : coder pour obtenir le rendu demandé, sans coder de HTML.

- chaque carte a une hauteur de 150px et une largeur de 300px,
- les cartes occupent toute la largeur du parent,
- les cartes se placent à la ligne si la fenêtre est trop petite,
- les cartes sont espacées de 10px,
- l'espace libre restant est réparti entre les cartes.

```
<div class="cartes">
  <div class="carte">Carte 1</div>
  <div class="carte">Carte 2</div>
  <div class="carte">Carte 3</div>
  ...
  <div class="carte">Carte 9</div>
  <div class="carte">Carte
10</div>
</div>
```

Projet : un mini-site

Énoncé

Vous devez créer un mini-site web.

- Votre site pourra être, au choix :
 - un site d'association (sportive, culturelle, humanitaire...),
 - un site événementiel (festival, tournoi, exposition, concert...),
 - un site touristique (ville, lieu, région, parc...).
- Le site contient trois pages au format HTML :
 - une page d'accueil `index.html`,
 - une page de contenu (bien choisir son nom),
 - une page `contact.html` ou `infos.html`.
- Un seul fichier `style.css` est utilisé par toutes les pages.
- La navigation est identique sur toutes les pages.
- Il y a des liens animés au survol.

Conseils de réalisation

AVANT DE CODER :

- Trouver sur Internet au moins 5 images adaptées à votre thème.
- Dessiner le site sur papier :
 - chaque page sur une feuille,
 - dessiner les différents éléments des pages,
 - nommer les zones par les balises utilisées.
- Dessiner la navigation en détail.

ORGANISER LE SITE EN HTML :

- Structurer les pages avec les balises adaptées.
- Commenter le code.
- Créer des contenus variés :
 - listes (activités, programme, services...),
 - images (lieux, affiches, photos...),
 - tableaux (horaires, tarifs, planning...).

METTRE EN FORME EN CSS :

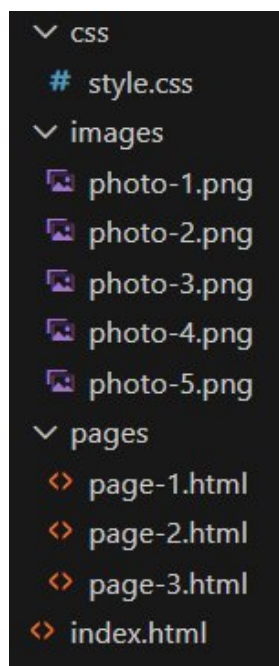
- Utiliser les classes.
- Choisir une police lisible.
- Harmoniser l'ensemble (couleurs, taille de texte, alignements).
- Utiliser `Flexbox` pour la navigation et / ou des cartes ou blocs côte à côte.
- Travailler les boîtes (marges extérieures et intérieurs, bordurer, arrière-plans).
- Effets en quantité modérés (coins arrondis, ombres, transparence).

HTML & CSS – Concepts avancés

Organiser un projet complet

Utilisation de plusieurs dossiers

Il est conseillé de regrouper les fichiers dans des dossiers. Cela nécessite d'adapter les chemins d'appel des fichiers, en parcourant l'arborescence. Le fichier `index.html` reste toujours à la racine.



- Pour `index.html` :

appel : <code>style.css</code>	<code><link rel="stylesheet" href="./css/style.css"></code>
insertion : image	<code></code>
lien : <code>page-1.html</code>	<code>Mon lien</code>
lien : <code>page-2.html</code>	<code>Mon lien</code>

- Pour `page-1.html` :

appel : <code>style.css</code>	<code><link rel="stylesheet" href="../css/style.css"></code>
insertion : image	<code></code>
lien : <code>index.html</code>	<code>Mon lien</code>
lien : <code>page-2.html</code>	<code>Mon lien</code>

- Le chemin des liens est toujours indiqué par rapport au fichier lui-même.
- Dans le parcours de l'arborescence, `./` indique le chemin du dossier dans lequel est le fichier.
- Dans les chemins de fichiers, `../` permet de remonter un niveau dans l'arborescence.

Bonnes pratiques de nommage

Les noms doivent être les plus explicites possibles, autant pour les dossiers, les fichiers et les classes. Il existe quelques conventions de nommage : minuscules, sans espace, sans accent et sans caractère spéciaux ; on sépare les mots avec "-" (kebab-case). Les chiffres sont possibles, mais jamais au début.

- Le nom d'un dossier doit correspondre aux fichiers qu'il contient (ex : `images`, `video`, ...).
- Le nom d'un fichier doit être cohérent avec son contenu (exception : `index.html`, obligatoire et réservé pour l'accueil).
- Le nom d'une classe doit explicitement exprimer le rôle des éléments appartenant à cette classe. La classe des sous éléments dérive du nom de la classe du bloc parent.

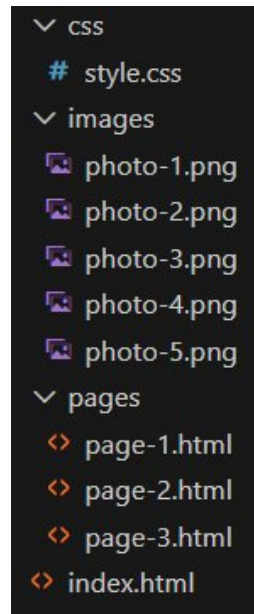
```
<nav class="navbar">
  <a class="navbar-link" href="#">...</a>
  <a class="navbar-link" href="#">...</a>
</nav>
```

Erreurs fréquentes

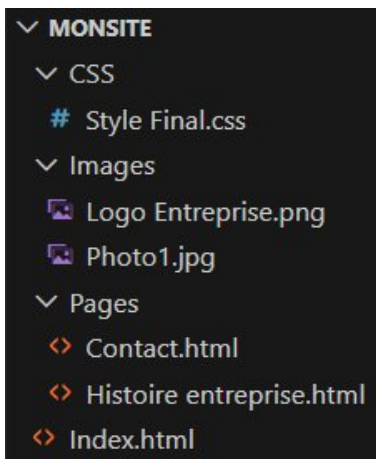
- Mettre tous les fichiers dans le même dossier.
- Nommer les dossiers ou les fichiers avec des caractères spéciaux, espace, majuscules.
- Changer de norme de nommage (`contact.html` et `Histoire.html` dans le même projet).
- Laisser des liens « cassés ».
- Utiliser des noms vagues.

Exercices

Lire du code : pour chaque fichier, identifier le dossier qui le contient.



Lire et écrire du code : corriger les erreurs de nommage.



```
<header>
  <link rel="stylesheet" href="CSS/Style Final.css">
</header>

<body>
  <header>
    <h1>Entreprise Exemple</h1>
  </header>

  <nav>
    <a href="Pages/Contact.html">Contact</a>
    <a href="Pages/Histoire entreprise.html">Histoire</a>
  </nav>

  <footer>
    <p>© Entreprise Exemple</p>
  </footer>
</body>
```

Écrire du code : coder pour obtenir le rendu demandé.

- Créer une arborescence de fichiers pour un site contenant :
 - une page d'accueil,
 - une page "services",
 - une page "contacts",
 - un fichier `style.css`,
 - plusieurs images.
- Coder :
 - l'appel du fichier CSS pour chaque page,
 - une navigation complète dans chaque page,
 - l'insertion d'une image dans chaque page.

Associer hiérarchie visuelle et cohérence graphique

Hiérarchie des titres

Les titres doivent refléter l'organisation du contenu.

- `<h1>` : titre principal (un seul par page),
- `<h2>` : grandes parties,
- `<h3>` : sous-parties.

La hiérarchie doit se refléter visuellement.

```
h1 {  
  font-size: 32px;  
}  
h2 {  
  font-size: 24px;  
}  
h3 {  
  font-size: 18px;  
}
```

Choix raisonné des couleurs

Les couleurs doivent être peu nombreuses, cohérentes entre elles, et servir la lisibilité. Conseil :

- 1 couleur principale,
- 1 couleur secondaire,
- 1 couleur pour le texte.

```
body {  
  color: #222;  
}  
h1, h2 {  
  color: darkblue;  
}  
h3, h4 {  
  color: lightblue;  
}
```

Gestion des contrastes et de la lisibilité

Un texte doit être lisible sans effort. À privilégier :

- texte foncé sur fond clair,
- contraste suffisant,
- taille de texte adaptée.

```
section {  
  background-color: #f2f2f2;  
  color: #000;  
  font-size: 18px;  
  line-height: 1.5em;  
}
```

Gestion des espacements

Les espaces structurent la page autant que les couleurs. On utilise :

- `margin` pour séparer les blocs,
- `padding` et `line-height` pour aérer le contenu.

```

section {
  margin: 30px;
  padding: 15px;
}

```

Erreurs fréquentes

- Tous les titres ont la même taille.
- Trop de couleurs différentes.
- Texte trop petit ou trop serré.
- Manque d'espace entre les sections.
- Espacements incohérents d'une page à l'autre.

Exercices

Lire du code : analyser un site et repérer les différents titres.



Lire et écrire du code : améliorer la lisibilité du site en modifiant le CSS.

```

body {
  color: lightgray;
  background-color: white;
}
section {
  margin: 5px;
}

```

Écrire du code : coder pour obtenir le rendu demandé.

- Écrire le CSS pour une page qui contient :
 - un titre principal,
 - plusieurs sections,
 - du texte long.
- Les objectifs sont la lisibilité du texte, la mise en évidence des titres, la gestion harmonieuse des espacements, des choix de couleurs sobres.

Maîtriser les effets visuels

L'interaction utilisateur avec `:active`

Le pseudo-sélecteur `:active` s'applique lorsqu'un utilisateur clique sur un élément (clic maintenu).

```
a {
  color: black;
  text-decoration: none;
}
a:active {
  color: red;
}
```

`:active` n'agit pas comme `:hover` :

- `:hover` : quand l'utilisateur survole l'élément, il voit que l'élément est interactif.
- `:active` : quand l'utilisateur clique sur l'élément, il voit que son action est prise en compte.

```
a:hover {
  color: blue;
}
a:active {
  color: red;
}
```

Utilisation d'un effet ... ou pas

Un effet visuel n'est pas obligatoire, et un site lisible sans effet est préférable à un site surchargé. Avant d'ajouter un effet, il faut se poser les questions suivantes :

- L'effet améliore-t-il la compréhension ?
- Est-il cohérent avec le reste du site ?
- Reste-t-il lisible et sobre ?

Si la réponse à l'une de ces questions est « non », alors il ne faut pas utiliser l'effet.

Erreurs fréquentes

- Effet trop visible ou trop agressif.
- Multiplier les effets sans raison.
- Différence trop faible entre l'état normal et `:active`.
- Modifier la taille ou la position d'un élément lors du clic.
- Utiliser `:hover` et `:active` sur des éléments non interactifs.

Exercices

Lire du code : décrire l'effet du code suivant.

```
button {
  background-color: lightgray;
  border: 1px solid black;
}
button:active {
  background-color: darkgray;
}
```

Lire et écrire du code : analyser le problème causé par ce code, puis proposer une amélioration.

```
a {  
  font-size: 15px;  
}  
a:active {  
  font-size: 30px;  
  margin-left: 20px;  
}
```

Écrire du code : coder pour obtenir le rendu demandé.

- un lien noir au repos,
- un lien bleu au survol,
- un lien gris foncé pendant le clic,
- un effet sobre et lisible.

```
<a href="#">Lien de navigation</a>
```

Utiliser les ressources externes

Polices Google

- On importe une [police Google](#) (ici, la police Roboto).

```
<head>  
  <link rel="preconnect" href="https://fonts.googleapis.com">  
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>  
  <link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap"  
rel="stylesheet">  
</head>
```

- On l'utilise ensuite en CSS avec `font-family`. On prévoit une police de secours (`sans-serif`).

```
body {  
  font-family: 'Roboto', sans-serif;  
}
```

Icônes Bootstrap

- On importe les [icônes Bootstrap](#) dans le `<head>`.

```
<head>  
  <link href="https://cdn.jsdelivr.net/npm/bootstrap-icons/font/bootstrap-icons.css"  
rel="stylesheet">  
</head>
```

- On intègre la bonne icône dans le `<body>` (ici, l'icône d'une enveloppe).

```
<a href="#"><i class="bi bi-envelope"></i>contact@entreprise.fr</a>
```

- On met en forme comme si c'était du texte en utilisant la classe de l'icône.

```
.bi-envelope {  
  color: blue;  
}
```

Erreurs fréquentes

- Multiplier les polices différentes (plus de 2).
- Choisir une police difficile à lire.
- Utiliser des icônes sans lien avec le contenu.
- Ne pas comprendre le code copié.
- Dépendre d'une ressource externe sans savoir à quoi elle sert.

Exercices

Lire du code : comprendre l'utilisation d'une ressource.

```
<link  
href="https://fonts.googleapis.com/css2?family  
=Lato&display=swap" rel="stylesheet">
```

```
h1 {  
    font-family: "Lato", sans-serif;  
}
```

- Quelle police est utilisée pour les titres ?
- Où est définie la police ?
- Que se passe-t-il si la police n'est pas chargée ?

Lire et écrire du code : corriger les erreurs du code suivant.

```
body {  
    font-family: Roboto;  
}
```

Écrire du code : coder pour obtenir le rendu demandé.

- Appliquer une police Google à tout votre site.
- Utiliser une icône Bootstrap pour illustrer une information de contact.

Projet : un site d'entreprise

Énoncé

Vous devez réaliser un site vitrine d'entreprise.

- Le site doit être professionnel, lisible et cohérent graphiquement.
- Il présentera une entreprise réaliste sur un thème au choix :
 - artisan (menuiserie, boulangerie, électricien, restaurant...),
 - petite entreprise locale,
 - start-up.
- Le site comporte 4 pages : accueil, produits, l'entreprise, contact.
- Il y a un seul fichier CSS pour toutes les pages.
- La navigation est identique sur toutes les pages.
- Les liens sont animés au survol et au clic.
- Il y a des liens vers les pages Facebook et X de l'entreprise, avec des icônes.
- Des polices Google et des icônes Bootstrap sont utilisés.

Conseils de réalisation

AVANT DE CODER :

- Trouver sur Internet au moins 5 images adaptées à votre thème.
- Organiser l'arborescence de votre projet (attention aux conventions de nommage).
- Déterminer vos choix de police, d'icônes, de couleurs.
- Dessiner le site sur papier.
- Dessiner la navigation en détail.

ORGANISER LE SITE EN HTML :

- Nommer les fichiers selon les conventions.
- Utiliser les balises sémantiques.
- Coder une navigation claire.
- Créer des contenus variés (listes, images, tableaux).

METTRE EN FORME EN CSS :

- Nommer les classes selon les conventions.
- Utiliser `Flexbox` au moins une fois.
- Utiliser les effets de votre choix de manière efficace.

HTML - cheatsheet

Base de fichier HTML

```
<!doctype html>      <!-- reconnaissance du code HTML5 par le navigateur -->
<html lang="fr">      <!-- langue du contenu de la page -->
  <head>               <!-- en-tête de la page (paramétrage) -->
    <meta charset="utf-8"> <!-- affichage correct de certains caractères -->
    <link rel="stylesheet"> <!-- lien vers un fichier CSS -->
    <title>...</title>  <!-- titre de l'onglet -->
  </head>
  <body>               <!-- contenu de la page -->
    ...
  </body>
</html>
```

Structurer une page

en-tête (page, section)	<header></header>
menu de navigation	<nav></nav>
contenu principal	<main></main>
section	<section></section>
article	<article></article>
contenu secondaire	<aside></aside>
pied (page, section)	<footer></footer>

Organiser un plan

titres (niveaux hiérarchiques)	
paragraphe	
retour à la ligne	

Balises neutres

bloc	
inline	

Créer une liste

liste à puces	
liste numérotée	

Créer un tableau

tableau	<table></table>
lignes	<tr></tr>
titre	<th></th>
cases	<td></td>

Indice, exposant

exposant	
indice	

Insérer des liens

page interne	Langage CSS
page externe	Mon ENT
nouvel onglet	Wikipedia
adresse mail	M'écire
téléphone	06 00 00 00 00

Insérer des médias

image	
audio	<audio controls src="/fichier.mp3"></audio>
vidéo	<video controls> <source src="/fichier.mp4" type="video/mp4"> </video>

Aide en ligne

<https://developer.mozilla.org/fr/docs/Web/HTML>

CSS – cheatsheet

Lier le fichier .css au fichier .html

```
<link rel="stylesheet" href="style.css"> <!-- à placer dans <head> -->
```

Sélecteurs

type	p {...}
classe	.classe-a {...}
multiple	.classe-a, p {...}
combiné (descendant)	div p {...}
combiné (enfant direct)	.classe-c > p {...}

Bordures (border)

épaisseur	border-width:	valeur
style de trait	border-style:	solid dotted dashed double
couleur	border-color:	couleur
coins arrondis	border-radius:	valeur

Texte

couleur	color:	couleur
alignement	text-align:	left right justify center
soulignement	text-decoration:	none underline
alinéa	text-indent:	valeur

Tableaux

cellule	border-collapse:	collapse separate
---------	------------------	----------------------

Arrière-plan (background)

couleur	background-color:	couleur	
image	background-image:	url(image)	
	background-repeat:	repeat no-repeat	
	background-attachment:	scroll fixed	
	background-position:	center top bottom left right	+ valeur
	background-size:	contain cover valeur	

Marges

marges extérieures	margin:	-left -right	valeur auto
marges intérieures	padding:	-top -bottom	valeur

Taille

largeur	width:	valeur fit-content
hauteur	height:	valeur fit-content

Police (font)

police	font-family:	polices
taille	font-size:	valeur
interligne	line-height:	valeur
italique	font-style:	normal italic
gras	font-weight:	normal bold

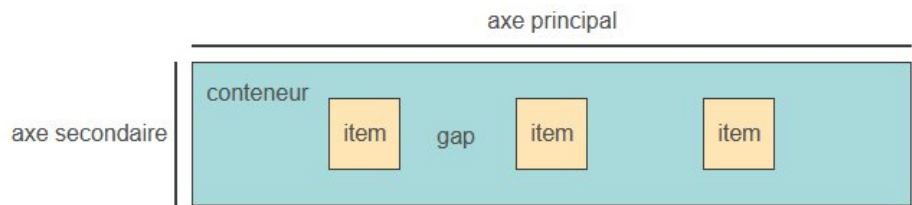
Listes

puces		disc circle square
numéros	list-style:	decimal upper-roman lower-roman lower-alpha upper-alpha

Effets spéciaux

ombre (texte)	text-shadow:	valeur valeur couleur
ombre (élément)	box-shadow:	valeur valeur couleur
opacité (objet)	opacity:	valeur (0 → 1)

Flexbox



Toutes ces propriétés portent sur le conteneur.

initialisation	display:	flex	
espacement	gap:	valeur	
retour à la ligne	flex-wrap:	nowrap	
		wrap	
axe principal	flex-direction:	row	
		column	
distribution selon l'axe principal	justify-content:	flex-start	
		flex-end	
		center	
		space-between	
		space-around	
		space-evenly	
		stretch	nécessite : <pre>.item { flex: 1; }</pre>
distribution selon l'axe secondaire	align-items:	flex-start	
		flex-end	
		center	
		stretch	nécessite : <pre>.item { flex: 1; }</pre>

Couleurs

	Hexadécimal	RGB
	#000000	rgb(0, 0, 0)
	#ffffff	rgb(255, 255, 255)
	#808080	rgb(128, 128, 128)
	#ff0000	rgb(255, 0, 0)
	#00ff00	rgb(0, 255, 0)
	#0000ff	rgb(0, 0, 255)
	#ffff00	rgb(255, 255, 0)
	#ff00ff	rgb(255, 0, 255)
	#00ffff	rgb(0, 255, 255)
	#c06fe8	rgb(192, 111, 232)

Transparence

```
rgba(0, 0, 0, 0.5) /* 0 ⇔ transp, 1 ⇔ opaque. */
```

Hexadécimal ↔ RGB

<https://htmlcolorcodes.com/fr/>

Unités de mesure

taille fixe	px
taille variable selon la taille de l'élément parent	%
taille variable selon la police du parent	em

Aide en ligne

<https://developer.mozilla.org/fr/docs/Web/CSS>